

NEW STUDENT CAMPUS HELPER

PROJECT REPORT

Course: CSE1021 – Introduction to Problem Solving and Programming

Program: Integrated M.Tech in AI

School: SCAI

Faculty Name:
Venkateswara Rao T

Submitted by:
Priyanshi Jamlya

Registration no. : 26MIM10130

VIT Bhopal University

Academic Year: 2026–27

1. Introduction

New Student Campus Helper is a Python-based application designed to help new students find important campus places and services easily. The project provides useful information about different campus facilities and allows users to access this information through a simple menu-driven interface.

The application includes features such as viewing campus information, searching for places, finding available services, getting quick help, and viewing a campus summary. It also uses input validation to handle invalid choices and provide a better user experience.

The project is developed using Python and follows a modular approach, where different features are organized into separate modules. This makes the program easier to understand, test, and maintain.

The project demonstrates important problem-solving concepts such as functions, conditional statements, loops, lists, dictionaries, input handling, validation, and modular programming.

2. Problem Statement

New students may face difficulty in finding important places and services on a large campus. They may not know the location, purpose, or available services of facilities such as the library, medical room, cafeteria, and other campus facilities. Searching for this information manually may take additional time.

The New Student Campus Helper project provides a simple solution to this problem. It allows students to view campus information, search for places, find services, get quick help, and view a summary through a menu-driven Python application.

3. Objectives

- To provide useful campus information to new students.
- To help users search for important campus places.
- To help users find available campus services.
- To provide quick help for common campus-related needs.
- To display a summary of available campus facilities.
- To validate user input and handle invalid choices.

To apply Python problem-solving and modular programming concept

4. Project Features

4.1 Campus Information

Provides information about important campus places and their basic details.

4.2 Search Place

Allows users to search for a specific campus place and view its available information.

4.3 Service Finder

Helps users find available services according to their requirements.

4.4 Quick Help

Provides quick assistance for common campus-related needs.

4.5 Campus Summary

Displays a summary of the available campus information and facilities.

4.6 Validation

Checks user input and handles invalid choices to prevent incorrect program flow

5. System Design

The system design represents the overall structure and working of the New Student Campus Helper. Different diagrams are used to show the system architecture, workflow, user interactions, components, and sequence of operations.

5.1 System Architecture

The system architecture shows the overall structure of the application. It represents how the user interacts with the main menu and validation system and how different modules process the user's request and provide the final result.

System Architecture-New Student Campus Helper

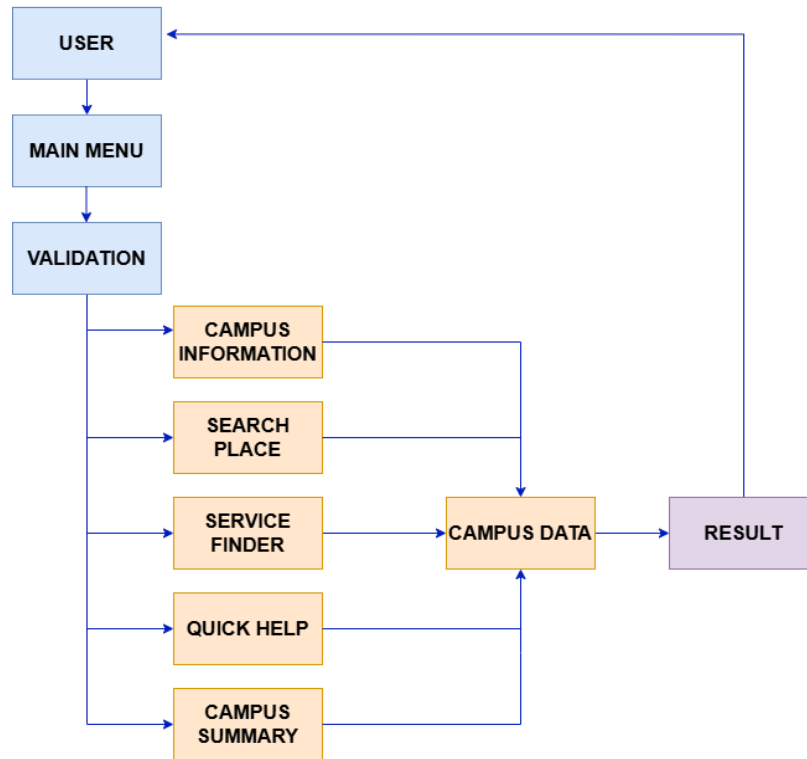


Figure 1: System Architecture of New Student Campus Helper

5.2 Workflow

The workflow diagram represents the sequence of steps followed by the application. It shows how the user's choice is taken, validated, processed, and followed by the appropriate result

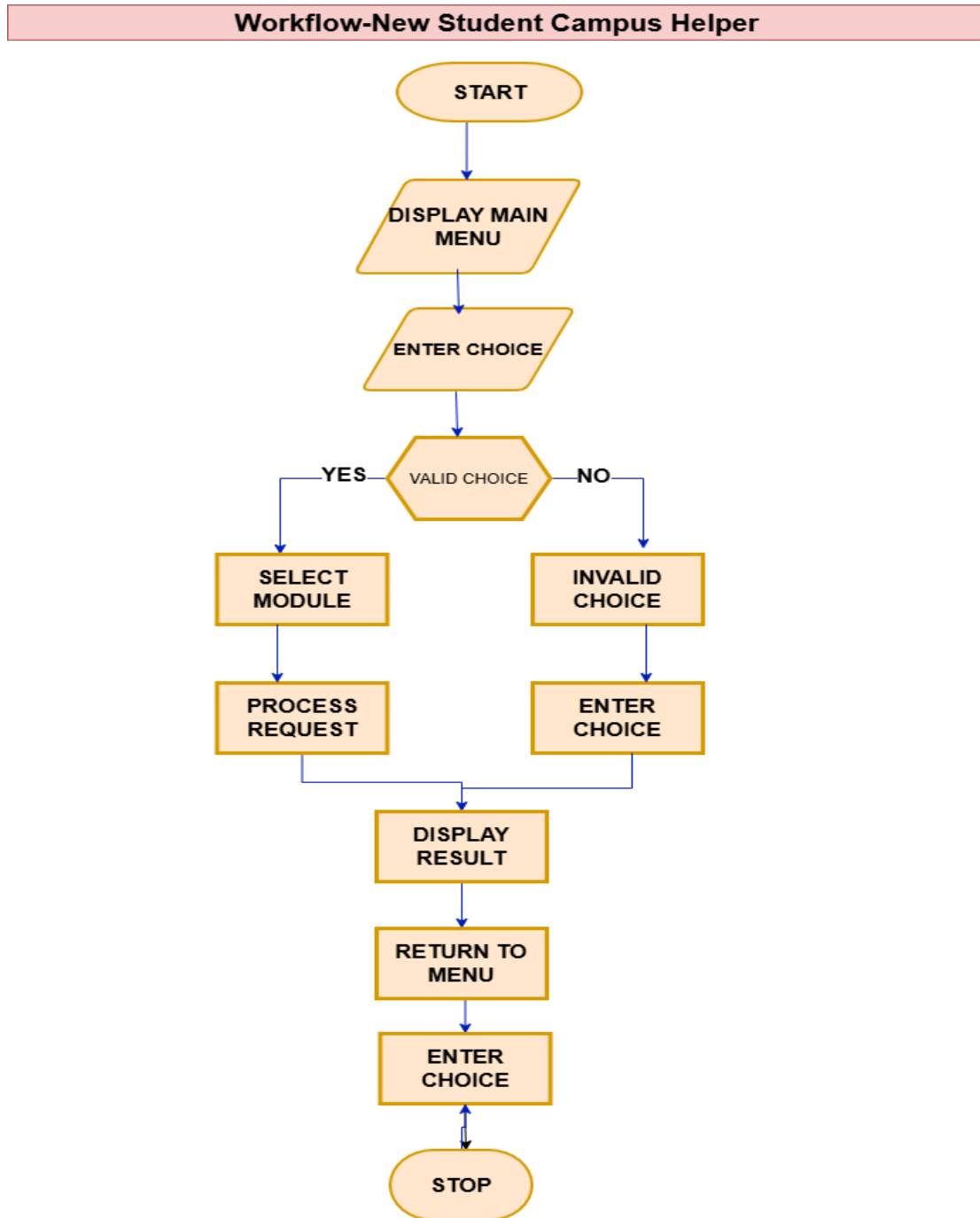


Figure 2: Workflow of New Student Campus Helper

5.3 Use Case Diagram

The use case diagram represents the main functions available to the user. It shows how the user interacts with the different features of the application.

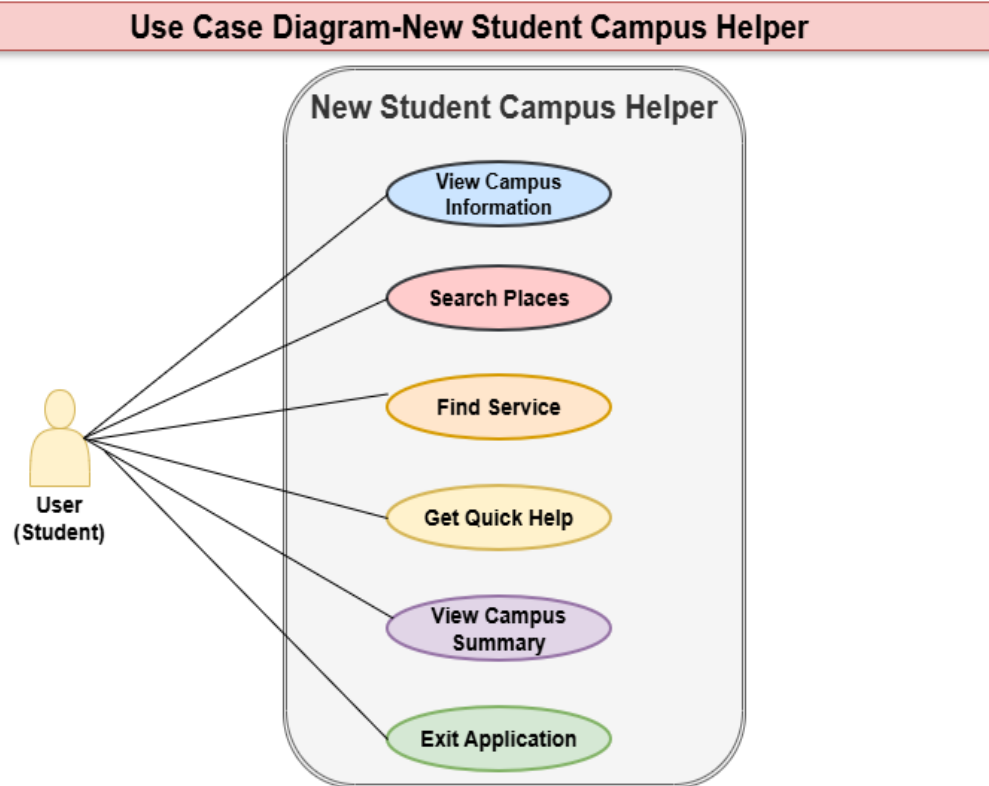


Figure 3: Use Case Diagram

5.4 Component Diagram

The component diagram represents the major modules of the project and their relationships. Each module performs a specific function and contributes to the overall working of the application.

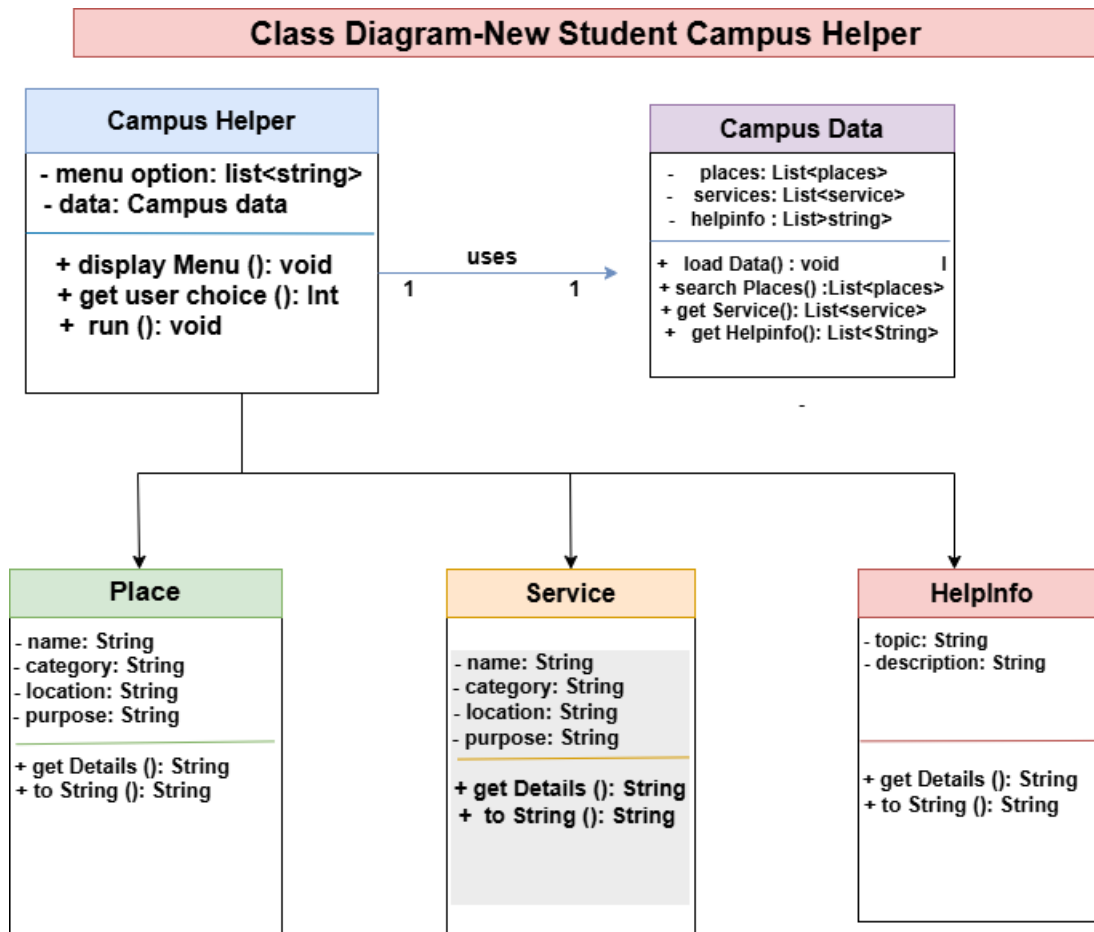


Figure 4: Component Diagram

5.5 Sequence Diagram

The sequence diagram shows the order of interaction between the user and the different parts of the system during a particular operation.

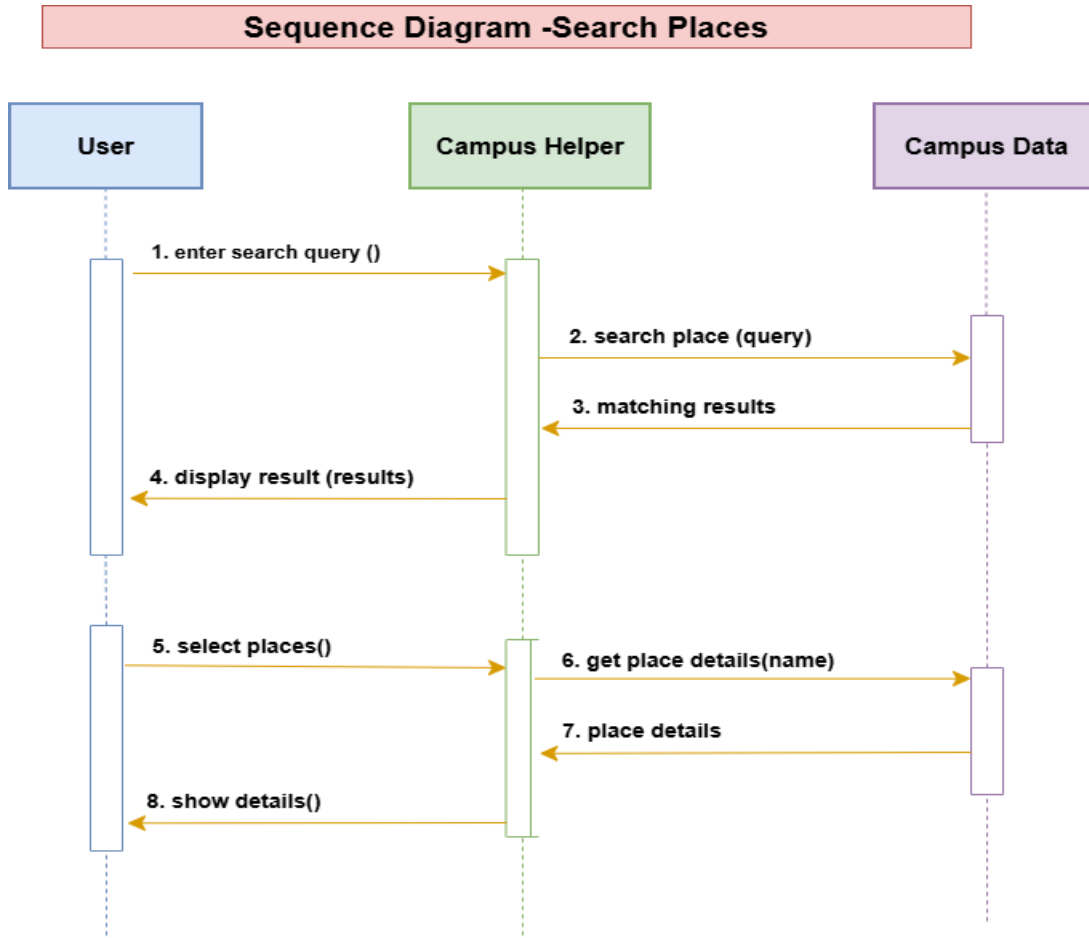


Figure 5: Sequence Diagram

6. Module Description

6.1 main.py

Controls the main menu and overall flow of the application. It connects the different modules and handles the user's selected option.

6.2 campus_data.py

Stores information about campus places, including their name, category, location, purpose, and services.

6.3 campus_info.py

Displays the available campus places and their basic information to the user.

6.4 search.py

Allows the user to search for a specific campus place and view its details.

6.5 service_finder.py

Helps the user find a campus place based on the service they need.

6.6 quick_help.py

Provides quick assistance for common needs such as study, books, and medical help.

6.7 summary.py

Displays a summary of the available campus places and categories.

6.8 validation.py

Checks whether the menu choice entered by the user is valid and helps handle invalid choices

7. Algorithm and Working

Algorithm

1. Start the application.
2. Display the main menu.
3. Take the user's choice.
4. Validate the entered choice.
5. Open the selected module.
6. Perform the requested operation.
7. Display the result.
8. Return to the main menu.
9. Continue until the user selects the exit option.

10. Stop the application.

The application follows a menu-driven approach. The user selects an option from the main menu, and the input is validated before the corresponding operation is performed. After displaying the result, the user can return to the main menu or exit the application

8. Implementation

The project was implemented using Python and divided into multiple modules. Each module is responsible for a specific feature of the application. Data is stored and organized using Python data structures, while functions, conditional statements, loops, and input validation are used to control the program flow.

8.1 Main Program

```
from campus_data import campus_places
from campus_info import display_places
from search import search_places
from service_finder import find_service
from quick_help import quick_help
from summary import show_summary
from validation import is_valid_menu_choice

while True:
    print("\n====New Student Campus Helper====")
    print("1. View Campus Places")
    print("2. Search Places")
    print("3. Find Service")
    print("4. Campus Summary")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if not is_valid_menu_choice(choice):
        print("Invalid choice.Please try again ")
        continue

    if choice == "1":
        display_places(campus_places)
```

Figure 6: Main Program Implementation (part 1)

```

if choice == "1":
    display_places(campus_places)

elif choice == "2":
    search_places(campus_places)

elif choice == "3":
    find_service(campus_places)

elif choice == "4":
    show_summary(campus_places)

elif choice == "5":
    quick_help()

elif choice == "6":
    print("Thank you for using Campus Heelper!")
    break

```

Main Program Implementation (part 2)

8.2 Campus Data

```

campus_places =[
    {
        "name": "Library",
        "category": "Academic",
        "location": "Main Academic Block",
        "purpose": "Books and study spaces",
        "services": {"Study", "Books", "Reading"}
    },
    {
        "name": "Medical Room",
        "category": "health",
        "location": "Main Campus",
        "purpose": "Basic Medical",
        "services": {"Medical Help", "first Aid"}
    }
]

```

Figure 7: Campus data implementation

8.3 Search Module

```
def search_places(campus_places):  
  
    search_name = input("Enter place name:")  
  
    found = False  
  
    for place in campus_places:  
        if place["name"].lower()==search_name.lower():  
            print("Name:",place["name"])  
            print("Category:",place["category"])  
            print("Location:",place["location"])  
            print("Purpose:",place["purpose"])  
            print("Services:",place["services"])  
            found = True  
  
    if found == False:  
        print("Place not found.")
```

Figure 8: Search module implementation

9. Testing

Testing was performed to check whether the different features of the application work correctly for valid and invalid inputs. The main functions were tested using different inputs and the obtained results were compared with the expected results.

9.1 Test Case 1 – View Campus Places

Input: 1

Expected Result: Campus places such as Library and Medical Room are displayed.

Actual Result: Campus places were displayed successfully.

Status: Pass

9.2 Test Case 2 – Invalid Choice

Input: 9

Expected Result: Invalid choice message is displayed.

Actual Result: Invalid choice message was displayed.

Status: Pass

9.3 Test Case 3 – Search Place

Input: 2 → Library

Expected Result: Library information is displayed.

Actual Result: Library information was displayed successfully.

Status: Pass

9.4 Test Case 4 – Non-existing Place

Input: 2 → Hostel XYZ

Expected Result: Place not found message is displayed.

Actual Result: Place not found message was displayed.

Status: Pass

9.5 Test Case 5 – Find Service

Input: 3 → [your available service]

Expected Result: Matching campus place/service information is displayed.

Actual Result: Matching information was displayed successfully.

Status: Pass

10. Testing Screenshots

The following screenshots show the actual execution of the project for different test cases. They demonstrate valid input, invalid input, place searching, handling of a non-existing place, and service finding

10.1 Valid Menu Choice

```
=====New Student Campus Helper=====
1. View Campus Places
2. Search Places
3. Find Service
4. Campus Summary
5. Exit
Enter your choice: 1

Name: Library
Category: Academic
Location: Main Academic Block
Purpose: Books and study spaces
Services: {'Reading', 'Books', 'Study'}

Name: Medical Room
Category: health
Location: Main Campus
Purpose: Basic Medical
Services: {'first Aid', 'Medical Help'}
```

Figure 9: Successful execution with valid input

10.2 Invalid Menu Choice

```
=====New Student Campus Helper=====
1. View Campus Places
2. Search Places
3. Find Service
4. Campus Summary
5. Exit
Enter your choice: 9
Invalid choice.Please try again
```

Figure 10: Handling of invalid input

10.3 Search Existing Place

```
=====New Student Campus Helper=====
1. View Campus Places
2. Search Places
3. Find Service
4. Campus Summary
5. Exit
Enter your choice: 2
Enter place name:library
Name: Library
Category: Academic
Location: Main Academic Block
Purpose: Books and study spaces
Services: {'Reading', 'Books', 'Study'}
```

Figure 11: Testing of search place feature

10.4 Search Non-existing Place

```
=====New Student Campus Helper=====
1. View Campus Places
2. Search Places
3. Find Service
4. Campus Summary
5. Exit
Enter your choice: 2
Enter place name:hostel XYZ
Place not found.
```

Figure 12: Handling of non-existing place

10.5 Find Service

```
=====New Student Campus Helper=====
1. View Campus Places
2. Search Places
3. Find Service
4. Campus Summary
5. Exit
Enter your choice: 3
Enter service:books
Service: books
Available at: Library
Location: Main Academic Block
```

Figure 13: Testing of service finder feature

11. Results

The project successfully provides the main campus assistance features. The application allows users to access campus information, search for places, find services, use quick help, and view a campus summary. Input validation also helps the application handle invalid choices.

12. Conclusion

The New Student Campus Helper project provides a simple Python-based solution for helping new students access useful campus information. The project demonstrates the application of problem-solving concepts, modular programming, data handling, user input, validation, and testing. The project also helped in understanding how a Python application can be divided into different modules for better organization and maintenance.

13. Future Scope

- More campus locations and services can be added.
- A graphical user interface can be developed.
- Campus maps and navigation can be integrated.
- Real-time campus information can be added.
- More search and filtering options can be provided.
- The application can be extended into a web or mobile application.

